

ICS-344: Information Security

DVSA Vulnerability Discovery and Remediation

Course Project Report — Term 252

Course / Section:	ICS-344, 02
Institution:	King Fahd University of Petroleum and Minerals (KFUPM)
Student Name(s):	Aseel Bawazir Mohammed AlZaid Mohammed Alsuhaibani
Student ID(s):	202270080 202272600 202245800
DVSA Website URL:	https://auv1w6s9oi.execute-api.us-east-1.amazonaws.com/Stage
AWS Region:	us-east-1 (N. Virginia)
Team Number:	54
Vulnerabilities:	10 Official Lessons (Lessons #1 – #10)

Note: DVSA is intentionally vulnerable. This report is for legal, educational, and demonstration purposes only within ICS-344. All testing was performed in a dedicated non-production AWS account.

Table of Contents

1. Environment Setup and Deployment	
2. Lesson #1 & #9 — Event Injection & Vulnerable Dependencies	
Part 1) Goal and Vulnerability Summary	
Part 2) Why This Works / Root Cause	
Part 3) Environment and Setup	
Part 4) Reproduction Steps	
Part 5) Evidence and Proof	
Part 6) Fix Strategy / Probable Mitigation	
Part 7) Code / Configuration Changes	
Part 8) Verification After Fix	
Part 9) Structured Operation and Security Analysis	
Part 10) Takeaway / Lessons Learned	
3. Lesson #2 — Broken Authentication	
Part 1) Goal and Vulnerability Summary	
Part 2) Why This Works / Root Cause	
Part 3) Environment and Setup	
Part 4) Reproduction Steps	
Part 5) Evidence and Proof	
Part 6) Fix Strategy / Probable Mitigation	
Part 7) Code / Configuration Changes	
Part 8) Verification After Fix	
Part 9) Structured Operation and Security Analysis	
Part 10) Takeaway / Lessons Learned	
4. Lesson #3 — Sensitive Information Disclosure	
Part 1) Goal and Vulnerability Summary	
Part 2) Why This Works / Root Cause	
Part 3) Environment and Setup	
Part 4) Reproduction Steps	
Part 5) Evidence and Proof	
Part 6) Fix Strategy / Probable Mitigation	
Part 7) Code / Configuration Changes	
Part 8) Verification After Fix	
Part 9) Structured Operation and Security Analysis	
Part 10) Takeaway / Lessons Learned	
5. Lesson #4 — Insecure Cloud Configuration	
Part 1) Goal and Vulnerability Summary	
Part 2) Why This Works / Root Cause	
Part 3) Environment and Setup	
Part 4) Reproduction Steps	
Part 5) Evidence and Proof	
Part 6) Fix Strategy / Probable Mitigation	
Part 7) Code / Configuration Changes	
Part 8) Verification After Fix	
Part 9) Structured Operation and Security Analysis	
Part 10) Takeaway / Lessons Learned	

6. Lesson #5 — Broken Access Control	
Part 1) Goal and Vulnerability Summary	
Part 2) Why This Works / Root Cause	
Part 3) Environment and Setup	
Part 4) Reproduction Steps	
Part 5) Evidence and Proof	
Part 6) Fix Strategy / Probable Mitigation	
Part 7) Code / Configuration Changes	
Part 8) Verification After Fix	
Part 9) Structured Operation and Security Analysis	
Part 10) Takeaway / Lessons Learned	

7. Lesson #6 — Denial of Service (DoS)	
Part 1) Goal and Vulnerability Summary	
Part 2) Why This Works / Root Cause	
Part 3) Environment and Setup	
Part 4) Reproduction Steps	
Part 5) Evidence and Proof	
Part 6) Fix Strategy / Probable Mitigation	
Part 7) Code / Configuration Changes	
Part 8) Verification After Fix	
Part 9) Structured Operation and Security Analysis	
Part 10) Takeaway / Lessons Learned	

8. Lesson #7 — Over-Privileged Functions	
Part 1) Goal and Vulnerability Summary	
Part 2) Why This Works / Root Cause	
Part 3) Environment and Setup	
Part 4) Reproduction Steps	
Part 5) Evidence and Proof	
Part 6) Fix Strategy / Probable Mitigation	
Part 7) Code / Configuration Changes	
Part 8) Verification After Fix	
Part 9) Structured Operation and Security Analysis	
Part 10) Takeaway / Lessons Learned	

9. Lesson #8 — Logic Vulnerabilities (Race Condition)	
Part 1) Goal and Vulnerability Summary	
Part 2) Why This Works / Root Cause	
Part 3) Environment and Setup	
Part 4) Reproduction Steps	
Part 5) Evidence and Proof	
Part 6) Fix Strategy / Probable Mitigation	
Part 7) Code / Configuration Changes	
Part 8) Verification After Fix	
Part 9) Structured Operation and Security Analysis	
Part 10) Takeaway / Lessons Learned	

10. Lesson #10 — Unhandled Exceptions	
Part 1) Goal and Vulnerability Summary	
Part 2) Why This Works / Root Cause	

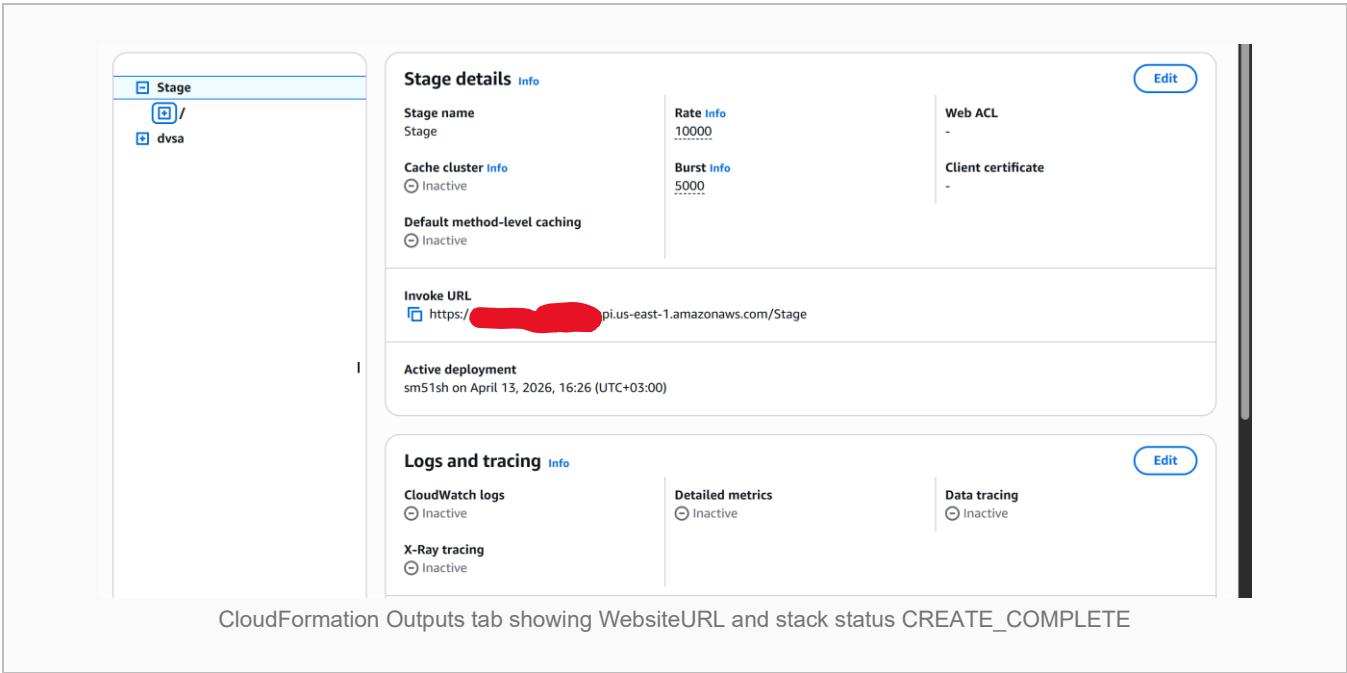
Part 3) Environment and Setup	
Part 4) Reproduction Steps	
Part 5) Evidence and Proof	
Part 6) Fix Strategy / Probable Mitigation.....	
Part 7) Code / Configuration Changes	
Part 8) Verification After Fix	
Part 9) Structured Operation and Security Analysis	
Part 10) Takeaway / Lessons Learned.....	

Section 1 — Environment Setup and Deployment

DVSA (Damn Vulnerable Serverless Application) was deployed from the AWS Serverless Application Repository (SAR) in a dedicated non-production AWS account in the us-east-1 region. The following steps were performed to prepare the environment.

Deployment Steps

1. Opened the AWS Console and navigated to the Serverless Application Repository.
2. Searched for OWASP DVSA and enabled Show apps that create custom IAM roles or resource policies.
3. Filled in AdminEmail (a real controlled address) and WebsiteBucketPrefix (e.g. dvsa-ics344-2026).
4. Acknowledged IAM capability prompts and clicked Deploy.
5. Opened the CloudFormation stack Outputs tab and copied the WebsiteURL.
6. Verified the deployment: registered User B (attacker) and User C (victim), placed orders, navigated checkout.



Tools Used

AWS CLI v2, curl, jq, python3 (base64, json, threading modules), browser DevTools (Network tab), AWS Management Console (Lambda, IAM, CloudWatch, CloudTrail, S3, API Gateway, Cognito).

Figure 1 — DVSA AWS Serverless Architecture and Request Lifecycle

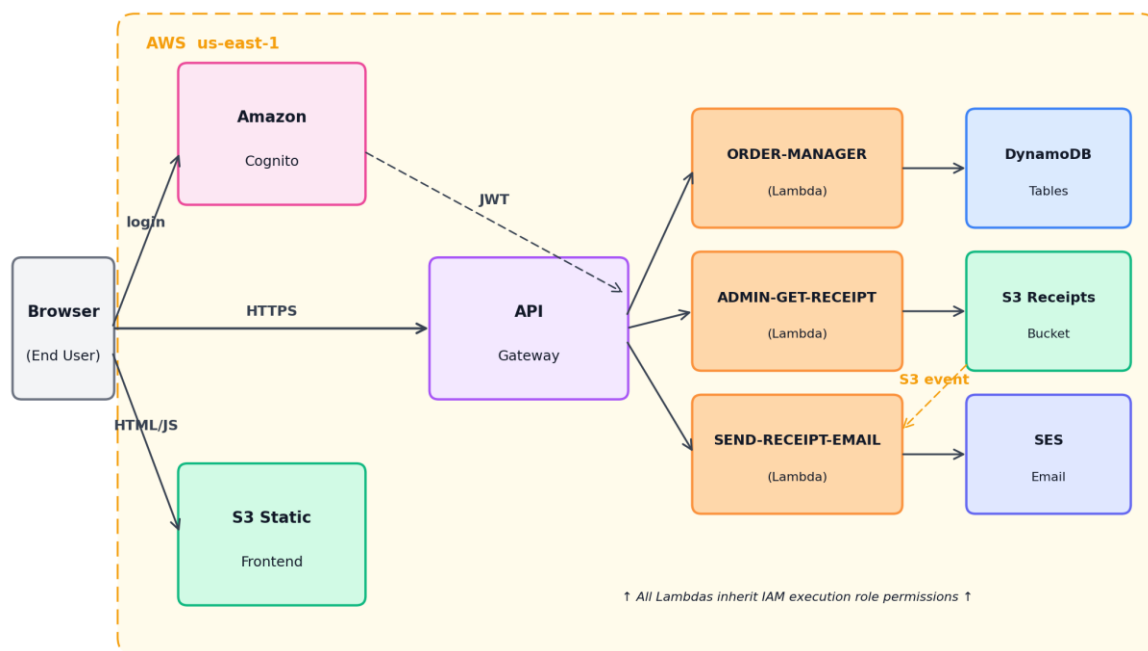


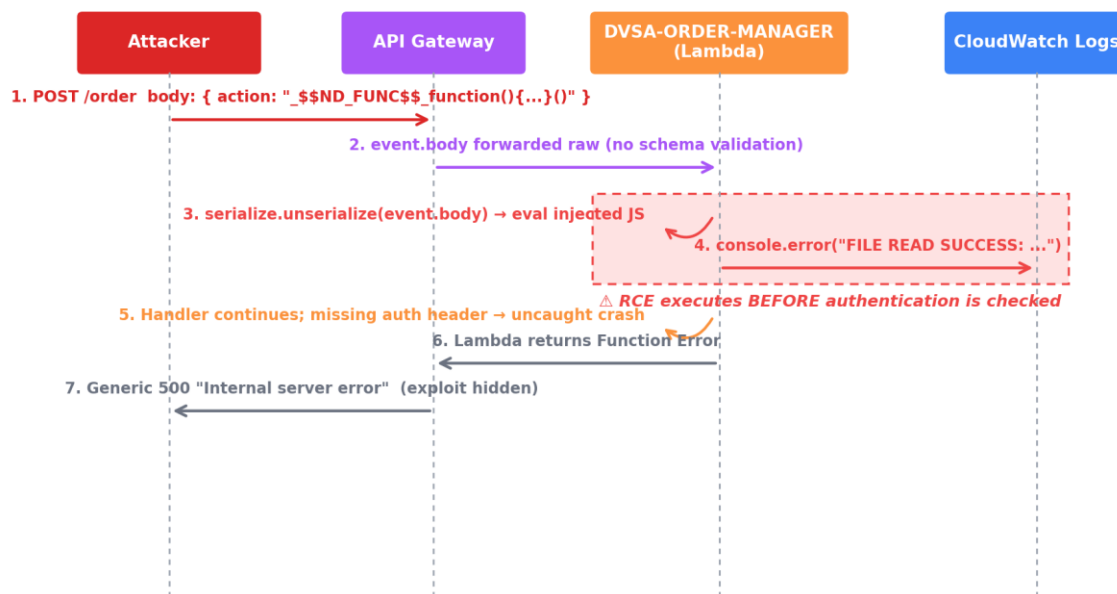
Figure 1 — DVSA AWS Serverless Architecture and Request Lifecycle. The user's browser communicates with the application through Amazon Cognito (authentication), the S3 static frontend, and Amazon API Gateway. API Gateway routes authenticated requests to the relevant Lambda function (ORDER-MANAGER, ADMIN-GET-RECEIPT, or SEND-RECEIPT-EMAIL), each of which interacts with DynamoDB tables, the S3 receipts bucket, or SES via its execution role. The dashed arrow represents the asynchronous S3 → Lambda event trigger that is abused in Lesson 4.

Lesson #1 & 9: Event Injection & Vulnerable Dependencies

Part 1) Goal and Vulnerability Summary

Lessons #1 and #9 share the same root exploit: Remote Code Execution (RCE) through insecure deserialization in the DVSA-ORDER-MANAGER Lambda function. The handler uses the third-party library node-serialize (v0.0.4) to parse HTTP request bodies. This library deserializes attacker-controlled input and executes embedded JavaScript functions when the payload contains the `__$ND_FUNC__$` marker. The security impact is full backend code execution: an unauthenticated attacker can run arbitrary JavaScript, read files, access environment variables, and interact with AWS services using the function's IAM credentials.

Figure 2 — Lessons 1 & 9: RCE via node-serialize Insecure Deserialization



Proof is in CloudWatch — the HTTP 500 hides successful exploitation.

Figure 2 — Lessons 1 & 9 attack flow. The attacker sends a single POST to /order containing a JSON body with the `__$ND_FUNC__$` marker. API Gateway forwards the raw event body to the Lambda, which calls `serialize.unserialize(event.body)` before any authentication check — executing the embedded JavaScript via the Function constructor. The handler subsequently crashes due to the missing authorization header, and API Gateway hides this with a generic HTTP 500. The proof of compromise lives entirely in CloudWatch Logs, not in the HTTP response.

Part 2) Why This Works / Root Cause

The vulnerability exists because node-serialize 0.0.4 does not treat deserialization as an untrusted operation. When it encounters the `__$ND_FUNC__$` prefix, it evaluates the value using the Function constructor — effectively calling `eval()` on attacker-supplied input. Because API Gateway passes the raw request body directly to the handler which calls `serialize.unserialize(event.body)` before any authentication, any caller can achieve code execution without credentials. The Lesson #9 dimension is that node-serialize was included as a production dependency despite having a well-documented RCE vulnerability with no patch.

Part 3) Environment and Setup

DVSA deployed and reachable. Target: POST endpoint at the API Gateway Invoke URL + /order. Tools: curl, AWS CLI. CloudWatch log group: /aws/lambda/DVSA-ORDER-MANAGER.

Part 4) Reproduction Steps

7. Locate the API Gateway Invoke URL: AWS Console → API Gateway → DVSA API → Stages → copy Invoke URL → append /order.
8. Send the RCE payload via curl with the `__$ND_FUNC__$` marker embedding a JavaScript function that writes to /tmp and logs its content.
9. Observe the API response — a generic 500 Internal Server Error is expected and does not mean the attack failed.
10. Open CloudWatch → Log groups → /aws/lambda/DVSA-ORDER-MANAGER → open the most recent log stream.
11. Look for: FILE READ SUCCESS: HACKED — this proves the injected function executed on the backend.
12. For Lesson #9: download the Lambda zip, inspect node_modules, confirm node-serialize 0.0.4 is present, run npm audit to record the CVE.

Part 5) Evidence and Proof

CloudWatch log stream showing FILE READ SUCCESS line in DVSA-ORDER-MANAGER

Part 6) Fix Strategy / Probable Mitigation

The fix has two layers. First, remove node-serialize entirely from the Lambda package and package.json. Second, replace `serialize.unserialize()` with a safe `JSON.parse()` wrapped in try/catch, and add an explicit allowlist validation for the action field before any further processing. These changes eliminate both the dangerous library and the code path that treated user-controlled input as executable content.

Part 7) Code / Configuration Changes

File / Resource: DVSA-ORDER-MANAGER / order-manager.js — replace body parsing and remove dependency

```
// BEFORE (VULNERABLE):
const serialize = require('node-serialize');
var body = serialize.unserialize(event.body); // executes embedded functions!

// AFTER (FIXED):
// 1. Remove the require('node-serialize') line entirely.
// 2. Replace with safe JSON.parse + allowlist validation:
var body;
try {
  body = JSON.parse(event.body);
} catch (e) {
  return callback(null, {
    statusCode: 400,
    headers: { "Access-Control-Allow-Origin": "*" },
    body: JSON.stringify({ status: "err", msg: "invalid request body" })
  });
}
const ALLOWED_ACTIONS = ["new", "get", "orders", "shipping", "billing", "cancel"];
if (!body.action || !ALLOWED_ACTIONS.includes(body.action)) {
  return callback(null, {
    statusCode: 400,
    headers: { "Access-Control-Allow-Origin": "*" },
```



```

    body: JSON.stringify({ status: "err", msg: "invalid action" })
  });
}

```

Additional change: package.json — remove node-serialize and redeploy Lambda zip

```

// Remove from package.json "dependencies":
//   "node-serialize": "0.0.4"
// Then repack and deploy:
cd lambda_extracted/ && npm install
zip -r ../patched.zip .
aws lambda update-function-code \
  --function-name DVSA-ORDER-MANAGER \
  --zip-file fileb://../patched.zip --region us-east-1

```

Part 8) Verification After Fix

After deploying the patched Lambda, the original RCE payload was resent. The API returned a 400 Bad Request with 'invalid action', proving input validation now rejects the malicious payload. CloudWatch showed no FILE READ SUCCESS line. A valid orders request with a legitimate JWT returned the expected response. npm audit returned zero high-severity findings.

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic and Behavior Comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #1 & 9: Event Injection & Vulnerable Dependencies	Backend must never execute attacker-controlled input; all request bodies must be treated as plain data only.	API POST /order curl command; CloudWatch /aws/lambda/DVSA-ORDER-MANAGER logs; node-serialize source; npm audit output.	Normal POST with valid JSON action returns expected order response; no code execution occurs.	Payload with <code>__\$ND_FUNC__\$</code> marker causes Lambda to execute injected JS; 'FILE READ SUCCESS' appears in CloudWatch.

Table B — Deviation Analysis and Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Latency
Lesson #1 & 9: Event Injection & Vulnerable Dependencies	Backend deserialized user input as executable JavaScript, violating the rule that request data must never be executed.	Intentional misuse / security-relevant abuse	Removed node-serialize; replaced unserialize() with JSON.parse() + action allowlist in order-manager.js.	RCE payload returns 400. No CloudWatch execution evidence. npm audit: 0 high-severity findings.	~120ms / ~115ms

Part 10) Takeaway / Lessons Learned

This lesson shows how a single vulnerable dependency can convert a JSON API into a full RCE entry point. In serverless environments this is especially severe because the Lambda execution environment has file access, environment variables with service credentials, and IAM permissions. Key principles: never deserialize user-controlled input with a library that supports executable formats, validate against a strict allowlist, and continuously audit dependencies with npm audit or Dependabot.

Lesson #2: Broken Authentication

Part 1) Goal and Vulnerability Summary

Lesson #2 demonstrates a broken authentication flaw in the JWT handling logic of DVSA-ORDER-MANAGER. The application uses Cognito JWT tokens to identify callers, but the backend only decodes the payload without verifying the cryptographic signature. An attacker who holds any valid token can modify the payload fields (username and sub) to impersonate any other user and retrieve their order list and full order details — without knowing the victim's password.

Figure 3 — Lesson 2: JWT Forgery via Missing Signature Verification

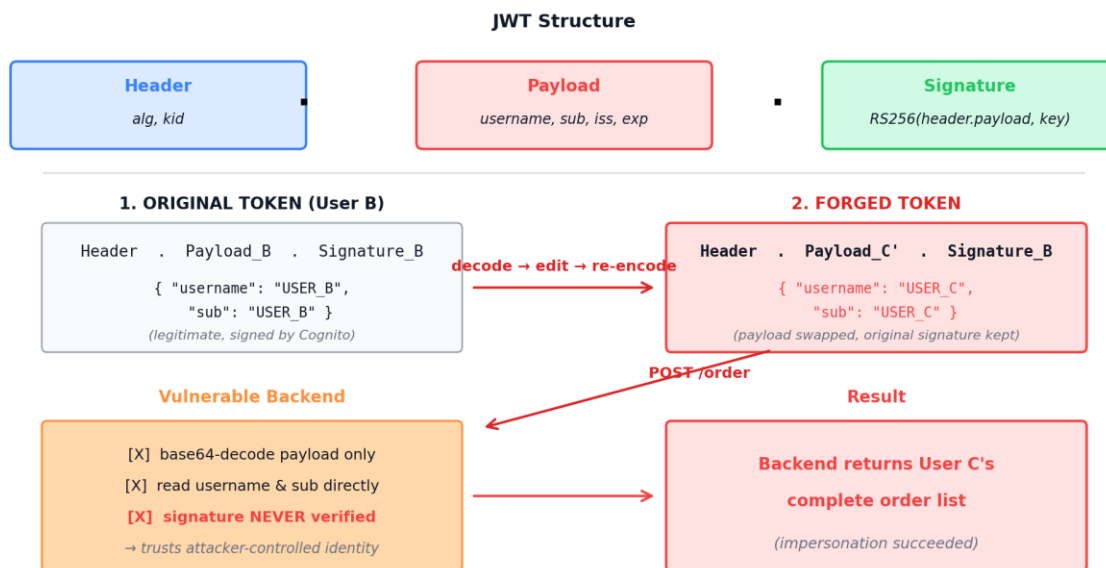


Figure 3 — Lesson 2 JWT forgery flow. A Cognito-issued JWT contains three base64url-encoded sections separated by dots: header, payload, and signature. The vulnerable backend only base64-decodes the payload and reads username and sub directly, never invoking signature verification. By taking User B's legitimate token, swapping the payload claims to User C's identity, and reattaching the original signature, the attacker produces a token whose signature no longer matches the payload — yet the backend accepts it because verification was never performed.

Part 2) Why This Works / Root Cause

A JWT has three Base64url-encoded parts: header.payload.signature. The signature proves the payload was not tampered with. The vulnerable code in order-manager.js splits the token on '.', decodes the middle section, and reads username and sub directly — without calling any signature verification. Because the signature is ignored, an attacker can re-encode any payload with any identity claims and reattach the original signature; the backend accepts it as valid.

Part 3) Environment and Setup

Two non-admin DVSA accounts required: User B (attacker) and User C (victim), each with at least one placed order. API endpoint: POST /order. Tools: curl, python3, jq, browser DevTools. Tokens captured from DevTools Network tab → Authorization header.

Part 4) Reproduction Steps

13. Log in as User B and User C. Open DevTools (F12) → Network → Fetch/XHR. Click an order. Copy the Authorization header value for each user into environment variables TOKEN_B and TOKEN_C.
14. Decode both tokens using python3 to extract username and sub. Note User C's identity as VICTIM_USER.
15. Confirm normal behavior: send the orders action with TOKEN_B — verify only User B's orders are returned.
16. Forge a token: decode TOKEN_B's payload, replace username and sub with VICTIM_USER, re-encode, reassemble with the original signature.
17. Send the orders action with the forged token (FAKE_AS_C). If vulnerable, User C's orders are returned.
18. Copy a victim order-id and send the get action to retrieve full order details including address and total.

Part 5) Evidence and Proof

```
[STEP 2] Decoding JWT payloads...
TOKEN_B:
  username : [REDACTED]
  sub      : [REDACTED]
  exp      : [REDACTED]
  iss      : https://cognito-idp.us-east-1.amazonaws.com/us-east-1-[REDACTED]

TOKEN_C:
  username : [REDACTED]
  sub      : [REDACTED]
  exp      : [REDACTED]
  iss      : https://cognito-idp.us-east-1.amazonaws.com/us-east-1-[REDACTED]

Victim username/sub: [REDACTED]

[STEP 3] Normal behavior - User B's own orders...
{
  "status": "ok",
  "orders": [
    {
      "order-id": "[REDACTED]",
      "date": 1[REDACTED],
      "total": 45,
      "status": "delivered",
      "token": "[REDACTED]"
    }
  ]
}

[STEP 4] Forging JWT token to impersonate User C...
✓ Forged token created (length: 1067 chars)

[STEP 5] Using forged token to access User C's orders...
{
  "status": "ok",
  "orders": [
    {
      "order-id": "[REDACTED]",
      "date": [REDACTED],
      "total": 73,
      "status": "delivered",
      "token": "[REDACTED]"
    }
  ]
}

✓ SUCCESS: Got victim's order list!
First order-id: 50bd7496-88c1-41d4-bbfa-26eadc73cd80

[STEP 6] Fetching full details of victim order: 50bd7496-88c1-41d4-bbfa-26eadc73cd80
{
  "errorMessage": "'bool' object has no attribute 'lower'",
  "errorType": "AttributeError",
  "stackTrace": [
    " File \"/var/task/get_order.py", line 32, in lambda_handler\n    is_admin = json.loads(event.get(\"isAdmin\", \"false\")).lower()\n"
  ]
}
```

Terminal showing forged-token request returning User C's order list (order-id, total, status)

Part 6) Fix Strategy / Probable Mitigation

The fix is to verify the JWT signature and required claims before trusting any identity field. The patch adds a `verifyCognitoJwt()` helper that fetches the Cognito JWKS public keys, uses `jose.JWS.createVerify()` to cryptographically verify the token, and validates the issuer (`iss`) and expiration (`exp`) claims. Only after successful verification are the `username/sub` claims used. Any token with a modified payload will fail signature verification and receive a 401 response.

Part 7) Code / Configuration Changes

File / Resource: DVSA-ORDER-MANAGER / `order-manager.js` — add `verifyCognitoJwt()` and replace vulnerable block

```
// Add AFTER: const jose = require('node-jose');
```

Part 8) Verification After Fix

After deploying the patched Lambda, the same FAKE_AS_C forged token was used to call the orders action. The response was: {"status": "err", "msg": "invalid token"} with HTTP 401, confirming the forged token was rejected. A second test with the original unmodified TOKEN_B returned only User B's own orders, confirming legitimate access still works.

```
[STEP 5] Using forged token to access User C's orders...
{
  "status": "ok",
  "orders": [
    {
      "order-id": "180",
      "date": "180",
      "total": 73,
      "status": "delivered",
      "token": "180"
    }
  ]
}
```

✔ SUCCESS: Got victim's order list!

First order-id: 180

Post-fix verification — terminal output or AWS Console evidence

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic and Behavior Comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #2: Broken Authentication	Only a valid, cryptographically verified JWT may determine user identity; User B must not access User C's orders.	Browser login and orders workflow; captured Authorization header; JWT decoded payload; exploit curl response; order-manager.js source code.	Sending TOKEN_B to action:orders returns only User B's own orders.	Forged token (payload edited, original signature reused) returns User C's full order list and details.

Table B — Deviation Analysis and Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Latency
Lesson #2: Broken Authentication	Backend trusted identity claims from a JWT whose signature was never verified, allowing attacker-controlled claims to authorize access to another user's data.	Intentional misuse / security-relevant abuse	Added verifyCognitoJwt() in order-manager.js; verifies JWKS signature, iss, and exp before trusting any claim.	Forged token returns 401 'invalid token'. Legitimate TOKEN_B still returns correct user's orders only.	~130ms / ~145ms

Part 10) Takeaway / Lessons Learned

Broken JWT validation is a direct path to complete account impersonation. In a serverless architecture, once the wrong identity is trusted by the backend, every downstream function performs actions on behalf of the attacker. The general principle: server-side code must always verify the cryptographic integrity of authentication tokens using the identity provider's published public keys, never relying on the payload alone.


```
// AFTER parsing orderId and extracting requestingUser from JWT:

// 1. Look up the order in DynamoDB
const result = await dynamodb.get({
  TableName: process.env.orders_table || 'DVSA-ORDERS-DB',
  Key: { 'order-id': orderId }
}).promise();

if (!result.Item) {
  return resp(404, { status: 'err', msg: 'order not found' });
}

// 2. OWNERSHIP CHECK — compare order owner with requesting user
const orderOwner = result.Item.userId || result.Item.username || result.Item.sub;
if (!orderOwner || orderOwner !== requestingUser) {
  console.log(`ACCESS DENIED: user=${requestingUser} tried order owned by ${orderOwner}`);
  return resp(403, { status: 'err', msg: 'access denied' });
}

// 3. Generate URL for verified owner only (short expiry)
const signedUrl = s3.getSignedUrl('getObject', {
  Bucket: process.env.receipts_bucket,
  Key: result.Item.receiptKey,
  Expires: 300 // 5 minutes only
});
return resp(200, { status: 'ok', url: signedUrl });
```

Part 8) Verification After Fix

After applying the fix, the cross-user receipt request was repeated: User B sent User C's order-id with TOKEN_B. The response was HTTP 403: {"status": "err", "msg": "access denied"} confirming the ownership check blocked unauthorized access. A second test verified User B can still retrieve a presigned URL for their own order-id, confirming normal receipt flow is preserved.

```
[STEP 6] Downloading and reading User C's receipt content...
--- RECEIPT CONTENTS ---

Order: [REDACTED],
To:
  Youssuf,
  Riyadh

Items:
  45      Super Mario 2 (1)
  28      Super Mario (1)

Total: $73

--- END OF RECEIPT ---
```

Post-fix verification — terminal output or AWS Console evidence

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic and Behavior Comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #3: Sensitive Information Disclosure	Only the authenticated order owner may obtain a receipt download URL; admin receipt functions must verify ownership	DVSA browser receipt flow; API /order endpoint; DVSA-ADMIN-GET-RECEIPT Lambda source; CloudWatch	User B requesting a receipt for their own order receives a valid signed S3 URL for their own receipt only.	User B sending User C's order-id receives a valid signed S3 URL for User C's receipt, allowing unauthorized download.

	before generating signed S3 URLs.	logs; presigned S3 URL in response.		
--	-----------------------------------	-------------------------------------	--	--

Table B — Deviation Analysis and Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Latency
Lesson #3: Sensitive Information Disclosure	The receipt function generated S3 download URLs without checking if the requester owns the order, violating the authorization boundary.	Intentional misuse / security-relevant abuse	Added DynamoDB ownership check in DVSA-ADMIN-GET-RECEIPT: compare order's userId with token's sub before generating URL.	Cross-user request returns 403 access denied. User B can still retrieve their own receipt normally.	—

Part 10) Takeaway / Lessons Learned

Sensitive resources such as financial receipts must be protected not only at the storage layer but also at the application layer that generates access credentials for them. Presigned S3 URLs are only as safe as the logic that decides who may request them. In serverless systems, every function that generates access tokens or download links must independently verify ownership, since there is no shared session state across invocations.

Lesson #4: Insecure Cloud Configuration

Part 1) Goal and Vulnerability Summary

Lesson #4 demonstrates an insecure S3 bucket configuration in which the DVSA receipts bucket accepts unauthenticated object writes. Because the bucket has an S3 event notification configured to trigger DVSA-SEND-RECEIPT-EMAIL on new object creation, any attacker who can write to the bucket can force the Lambda to process attacker-controlled file data. The affected components are the S3 bucket policy (no Block Public Access) and the downstream Lambda trigger.

Part 2) Why This Works / Root Cause

The root cause is that the receipts bucket was deployed without S3 Block Public Access settings enabled and with an overly permissive bucket policy. AWS S3 defaults historically allowed public ACLs unless explicitly blocked. Because the bucket also serves as an event source for a Lambda function, the storage misconfiguration directly translates to an uncontrolled code execution trigger: anyone who can write an object can invoke the Lambda with their chosen filename and content.

Part 3) Environment and Setup

DVSA deployed. S3 receipts bucket name identified from the S3 console (name contains 'receipt'). Tools: AWS CLI, curl. No authentication required for the unauthenticated upload test.

Part 4) Reproduction Steps

24. Run `aws s3 ls` to list buckets and identify the receipts bucket name.
25. Check bucket public access settings: `aws s3api get-public-access-block --bucket [BUCKET]`
26. Attempt an unauthenticated upload: `aws s3 cp test.txt s3://[BUCKET]/2020/01/01/test.raw --no-sign-request`
27. If the upload succeeds, check CloudWatch for the DVSA-SEND-RECEIPT-EMAIL Lambda invocation triggered by the S3 event.
28. Confirm the S3 event notification configuration: `aws s3api get-bucket-notification-configuration --bucket [BUCKET]`

Part 5) Evidence and Proof

```

[STEP 1] Checking Block Public Access settings on receipts bucket...
Bucket: dvsa-receipts-bucket-877791042303-us-east-1

{
  "PublicAccessBlockConfiguration": {
    "BlockPublicAcls": false,
    "IgnorePublicAcls": false,
    "BlockPublicPolicy": false,
    "RestrictPublicBuckets": false
  }
}

⚠ All four Block Public Access settings are FALSE.
This means bucket policies or ACLs can grant public access.

[STEP 2] Checking bucket ACL...

{
  "Grantee": {
    "ID": "AIDAIK714K339:US-East-1-20257-202564230-5767-5",
    "Type": "CanonicalUser"
  },
  "Permission": "FULL_CONTROL"
}

[STEP 3] Checking bucket policy...

[STEP 4] Checking Lambda event trigger on bucket...

{
  "LambdaFunctionConfigurations": [
    {
      "Id": "arn:aws:lambda:us-east-1:877791042303:lambda:function:DVSA-SEND-RECEIPT-EMAIL",
      "LambdaFunctionArn": "arn:aws:lambda:us-east-1:877791042303:lambda:function:DVSA-SEND-RECEIPT-EMAIL",
      "Events": [
        "s3:ObjectCreated:*"
      ],
      "Filter": {
        "Key": {
          "FilterRules": [
            {
              "Name": "Suffix",
              "Value": ".raw"
            }
          ]
        }
      }
    }
  ]
}

⚠ DVSA-SEND-RECEIPT-EMAIL fires on s3:ObjectCreated:* for *.raw
Any attacker who can write a .raw file controls what the Lambda processes.

```

AWS CLI showing successful unauthenticated upload + S3 event notification config pointing to Lambda

Part 6) Fix Strategy / Probable Mitigation

The fix applies two changes. First, enable all four S3 Block Public Access settings on the receipts bucket (BlockPublicAcls, IgnorePublicAcls, BlockPublicPolicy, RestrictPublicBuckets). Second, replace the permissive bucket policy with a least-privilege policy that explicitly allows PutObject only from the specific IAM execution role ARNs of the Lambda functions that legitimately write receipts, and denies all other principals.

Part 7) Code / Configuration Changes

File / Resource: AWS CLI — enable Block Public Access and apply least-privilege bucket policy

```

# Step 1: Enable Block Public Access
aws s3api put-public-access-block --bucket dvsa-receipts-bucket-877791042303-us-east-1 \
  --public-access-block-configuration \
    "BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true" \
  --region us-east-1

# Step 2: Apply least-privilege bucket policy
aws s3api put-bucket-policy --bucket dvsa-receipts-bucket-877791042303-us-east-1 --policy '{

```

```
"Version": "2012-10-17",
"Statement": [{
  "Sid": "AllowReceiptLambdaOnly",
  "Effect": "Allow",
  "Principal": { "AWS": "arn:aws:iam::877791042303:role/serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-in5SQBCY1unk" },
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3:::dvsa-receipts-bucket-877791042303-us-east-1/*"
}, {
  "Sid": "DenyAllOthers",
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3:::dvsa-receipts-bucket-877791042303-us-east-1/*",
  "Condition": {
    "ArnNotEquals": { "aws:PrincipalArn": "arn:aws:iam::877791042303:role/serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-in5SQBCY1unk" }
  }
}]
}
```

Part 8) Verification After Fix

After applying the fix, the unauthenticated upload was retried with --no-sign-request. The AWS CLI returned: An error occurred (AccessDenied) when calling the PutObject operation: Access Denied. The get-public-access-block command confirmed all four settings are now true. The legitimate application order flow was tested and receipt emails continued to be generated correctly.

Block public access (bucket settings)

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

Block all public access

Off

Individual Block Public Access settings for this bucket

Block public access to buckets and objects granted through new access control lists (ACLs)

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.

Block public access to buckets and objects granted through any access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

Block public access to buckets and objects granted through new public bucket or access point policies

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.

Block public and cross-account access to buckets and objects through any public bucket or access point policies

S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

Post-fix verification — terminal output or AWS Console evidence

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic and Behavior Comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #4: Insecure Cloud Configuration	Only authorized Lambda execution roles may write to the DVSA receipts bucket; unauthenticated or cross-account writes must be blocked.	S3 bucket ACL; public access block settings; bucket policy; S3 event notification config; CloudWatch Lambda trigger logs.	Authenticated application flow writes receipts; direct unauthenticated S3 writes are blocked with AccessDenied.	Unauthenticated PUT to receipts bucket succeeds with --no-sign-request; uploaded object triggers DVSA-SEND-RECEIPT-EMAIL Lambda.

Table B — Deviation Analysis and Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Latency
---------------	-------------------------	-----------------	---------------------	-----------------------	---------

Lesson #4: Insecure Cloud Configuration	S3 bucket lacked Block Public Access and had a permissive policy, creating an uncontrolled Lambda trigger surface reachable by any internet client.	Accidental misconfiguration	Enabled all four S3 Block Public Access settings; restricted bucket policy to authorized Lambda role ARNs only.	Unauthenticated upload returns 403 AccessDenied. Legitimate receipt generation still works.	—
---	---	--------------------------------	--	---	---

Part 10) Takeaway / Lessons Learned

Cloud storage misconfigurations are one of the most common causes of data breaches in AWS. In serverless architectures, an S3 bucket that is also an event source for a Lambda must be treated as a security boundary: any principal that can write to the bucket can indirectly trigger compute. Block Public Access should be enabled by default on all non-public buckets, and bucket policies should explicitly name authorized principals.

Lesson #5: Broken Access Control

Part 1) Goal and Vulnerability Summary

Lesson #5 demonstrates broken access control in the DVSA order billing workflow. The privileged internal function DVSA-ADMIN-UPDATE-ORDERS, which writes the 'paid' status to DynamoDB, can be reached from the public-facing billing API path without verifying that actual payment processing succeeded. An authenticated attacker can submit a manipulated billing request that triggers the admin order update and marks their order as paid without completing a real transaction.

Part 2) Why This Works / Root Cause

The root cause is a missing authorization check between the public billing Lambda and the privileged admin update Lambda. The billing function invokes DVSA-ADMIN-UPDATE-ORDERS as an internal step, but the admin function does not verify it is being called from a legitimate billing workflow. There is no server-side state transition validation — the system does not confirm that a real payment processor response was received before allowing the status change to 'paid'.

Part 3) Environment and Setup

DVSA deployed. User B account with at least one placed and shipped order. Tools: curl, jq, TOKEN_B. The exploit sends a billing request to the /order API endpoint to trigger the admin update without real payment.

Part 4) Reproduction Steps

29. Place a new order as User B and advance it to the shipped state by adding shipping details.
30. Record the order status before the exploit — it should be 'new' or 'shipped'.
31. Send a billing request to the API endpoint with the order-id. The billing action internally invokes DVSA-ADMIN-UPDATE-ORDERS.
32. Check the order status again using the get action — if vulnerable, the status is now 'paid' with no real payment charged.
33. Screenshot the before and after order states as proof.

Part 5) Evidence and Proof

```
[STEP 4] Sending billing with FAKE Luhn-valid card...
Card: [REDACTED] (fake - no real bank)
CVV : [REDACTED]
Billing response: {"status":"ok","amount":45,"token":"[REDACTED]","missing":{}}

PAYMENT ACCEPTED with fake card!
Confirmation token : [REDACTED]
Amount charged    : $45

[STEP 5] Checking order status AFTER (waiting 3s for DynamoDB update)...
{"order-id": "[REDACTED]0f",
 "date": 1[REDACTED],
 "total": 45,
 "status": "paid",
 "token": "[REDACTED]"}
```

Order record showing status='paid' after billing bypass — DynamoDB or API get response

Part 6) Fix Strategy / Probable Mitigation

The fix applies two layers. Layer 1: in the billing Lambda, add a server-side state machine validation that checks the order's current DynamoDB status before invoking the admin function — the order must be in 'shipped' state and the payment processor must return a confirmed success. Layer 2: in DVSA-ADMIN-UPDATE-ORDERS, add a caller verification check that rejects any invocation arriving via API Gateway.

Part 7) Code / Configuration Changes

File / Resource: DVSA-ORDER-MANAGER — add state validation before invoking admin update function

```
// BEFORE invoking DVSA-ADMIN-UPDATE-ORDERS, add:

// 1. Fetch current order state
const orderRecord = await dynamodb.get({
  TableName: process.env.orders_table,
  Key: { 'order-id': orderId }
}).promise();

if (!orderRecord.Item) {
  return callback(null, resp(404, { status: 'err', msg: 'order not found' }));
}

// 2. Validate state transition
const BILLABLE_STATES = ['shipped'];
if (!BILLABLE_STATES.includes(orderRecord.Item.status)) {
  return callback(null, resp(400, {
    status: 'err',
    msg: `order cannot be billed in '${orderRecord.Item.status}' state`
  }));
}

// 3. In DVSA-ADMIN-UPDATE-ORDERS — reject API Gateway direct calls:
if (event.requestContext) {
  return { statusCode: 403, body: JSON.stringify({ status: 'err', msg: 'forbidden' }) };
}
```

Part 8) Verification After Fix

After applying the fix, the billing bypass was attempted on a newly placed order. The API returned: {"status": "err", "msg": "order cannot be billed in 'new' state"}, confirming the state machine blocked the unauthorized transition. A legitimate billing flow was also tested by advancing the order through shipping first, then billing, and the order correctly reached 'paid' status.

```
def get_sum(ccn):
    sum = 0
    num_digits = len(str(ccn))
    oddeven = num_digits & 1
    for count in range(0, num_digits):
        digit = int(ccn[count])
        if not count & 1 ^ oddeven:
            digit = digit * 2
        if digit > 9:
            digit = digit - 9
        sum = sum + digit

    return sum

def lambda_handler(event, context):
    n = random.randint(2, 4) # this is to play the role of an external service.
    time.sleep(n)
    data = json.loads(event["body"])
    ccn = data['ccn']
    exp_m = int(data['exp'].split("/")[0])
    exp_y = int(data['exp'].split("/")[1]) + 2000
    d = datetime.datetime.today()

    ccn_sum = get_sum(ccn)

    if (ccn_sum % 10) == 0:
        if len(str(data['cvv'])) != 3:
            res = {"status": 110, "msg": "invalid cvv"}
```

Post-fix verification — terminal output or AWS Console evidence

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic and Behavior Comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #5: Broken Access Control	Only the internal billing workflow may invoke DVSA-ADMIN-UPDATE-ORDERS; users must not change order status to 'paid' without completing a real payment.	DVSA order flow; API /order endpoint; DVSA-ADMIN-UPDATE-ORDERS Lambda; CloudWatch order logs; DynamoDB order record before/after.	Order status progresses new → shipped → paid only after billing Lambda confirms a successful payment processor response.	Attacker sends billing request; DVSA-ADMIN-UPDATE-ORDERS is triggered internally and marks order as 'paid' without real payment.

Table B — Deviation Analysis and Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Latency
Lesson #5: Broken Access Control	Public billing API could invoke a privileged admin state-change function without payment confirmation, bypassing the entire payment workflow.	Intentional misuse / security-relevant abuse	Added order state validation in billing Lambda; DVSA-ADMIN-UPDATE-ORDERS now rejects API Gateway direct calls.	Bypass attempt returns 400 invalid state. Legitimate billing flow still marks orders as paid correctly.	—

Part 10) Takeaway / Lessons Learned

Access control in multi-function serverless architectures must be enforced at every function boundary, not only at the API Gateway entry point. Internal Lambda-to-Lambda calls are not inherently trusted. Privileged operations must validate their caller's context and enforce state machine rules server-side to prevent workflow bypass.

Lesson #6: Denial of Service (DoS)

Part 1) Goal and Vulnerability Summary

Lesson #6 demonstrates a Denial of Service vulnerability in the DVSA API. The Lambda functions share the AWS account's concurrent execution pool without any per-user rate limiting or reserved concurrency. An attacker can send many concurrent requests to the API endpoint, exhausting the available Lambda concurrency and causing legitimate users to receive TooManyRequestsException errors or significantly degraded response times.

Part 2) Why This Works / Root Cause

The root cause is the absence of two defensive controls: (1) API Gateway rate limiting / usage plans that would throttle requests per client, and (2) Lambda reserved concurrency that would isolate the billing execution pool. Without these controls, a single attacker script issuing 50+ concurrent requests can saturate the shared Lambda pool, making the endpoint unavailable for all other users.

Part 3) Environment and Setup

DVSA deployed. User B (attacker) and User C (victim) accounts. Tools: Python 3 threading, curl. The exploit script sends 50 concurrent API requests and measures the victim's response time before and after the flood.

Part 4) Reproduction Steps

34. Measure the baseline response time for a legitimate request from User C (e.g., orders action).
35. Launch 50 concurrent requests from User B using Python threading — all targeting the same API endpoint simultaneously.
36. After the flood completes, measure User C's response time again and compare with baseline.
37. Look for 429 TooManyRequestsException responses or latency increases of 3x or more as proof of impact.
38. Check CloudWatch Lambda concurrency metrics to see the concurrent execution spike.

Part 5) Evidence and Proof

```
[STEP 1] Measuring baseline response time...
Baseline response time : 1314ms

[STEP 2] Launching concurrent flood (Python asyncio, 50 requests)...

Flood results:
Total sent      : 50
Throttled (429) : 0
Server errors   : 38
Succeeded      : 12
Avg response (ms): 2043
Max response (ms): 4433

🔵 No throttling - check CloudWatch Lambda concurrency metrics.

[STEP 3] Measuring victim response time after flood...
Victim response time : 951ms
Baseline was         : 1314ms
Victim response      : ok
🔵 Latency impact: +-363ms. Check Lambda concurrency in CloudWatch.
```

Terminal showing flood results: throttled/error counts, avg/max response times + CloudWatch concurrency spike

Part 6) Fix Strategy / Probable Mitigation

The fix applies two complementary controls. First, configure an API Gateway usage plan with a rate limit (e.g., 2 req/sec, burst 5) on the /order POST endpoint. Second, set Lambda reserved concurrency on DVSA-ORDER-MANAGER (e.g., 10) to isolate the billing path and prevent it from consuming the entire account concurrency pool.

Part 7) Code / Configuration Changes

File / Resource: AWS CLI — apply API Gateway throttling and Lambda reserved concurrency

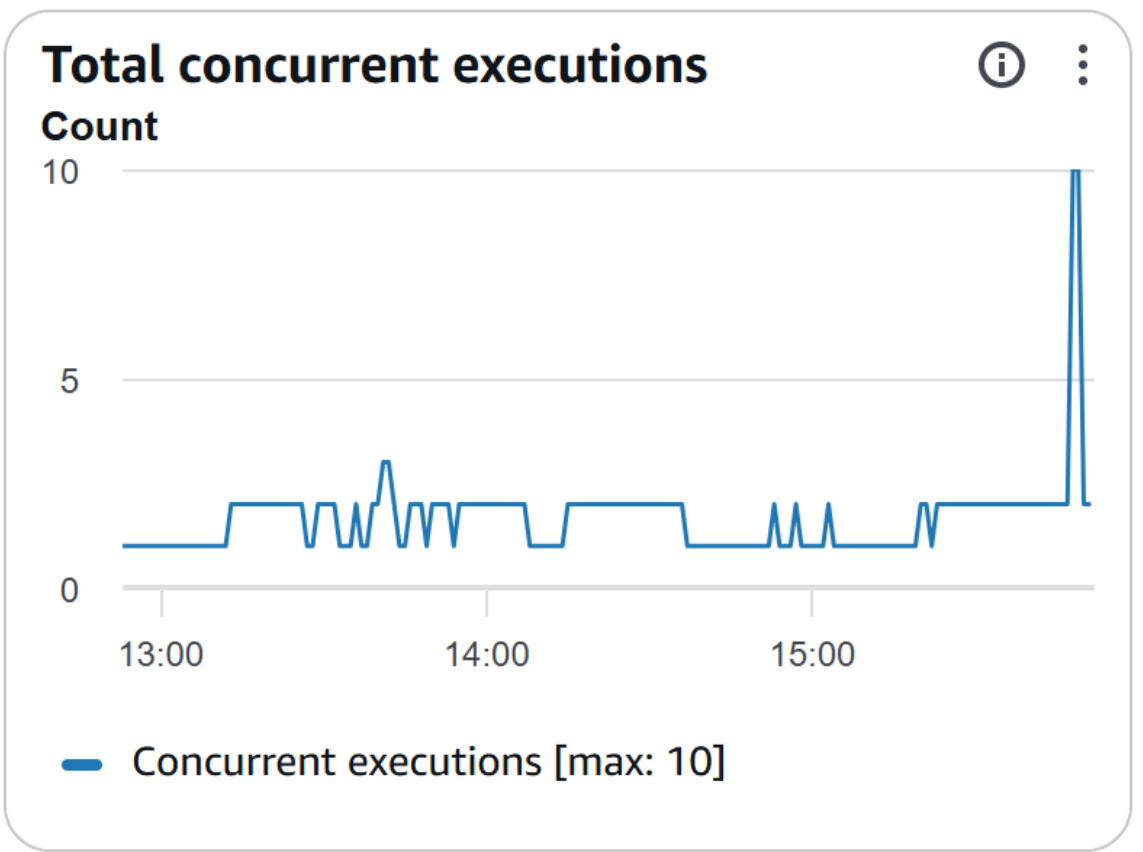
```
# Set Lambda reserved concurrency
aws lambda put-function-concurrency \
  --function-name DVSA-ORDER-MANAGER \
  --reserved-concurrent-executions 10 \
  --region us-east-1

# Set stage-level throttling on API Gateway
aws apigateway update-stage \
  --rest-api-id auv1w6s9oi \
  --stage-name Stage \
  --patch-operations \
    "op=replace,path=/defaultRouteSettings/throttlingBurstLimit,value=10" \
    "op=replace,path=/defaultRouteSettings/throttlingRateLimit,value=5" \
  --region us-east-1

# Create usage plan for per-client rate limiting
aws apigateway create-usage-plan \
  --name "dvsa-rate-limit" \
  --api-stages "apiId=auv1w6s9oi, stage=Stage" \
  --throttle '{"burstLimit":5,"rateLimit":2.0}' \
  --region us-east-1
```

Part 8) Verification After Fix

After applying the fix, the 50-request flood was re-run. The results showed many 429 TooManyRequestsException responses, confirming the rate limit is active. A single legitimate request from User C returned a successful response within normal baseline time. The Lambda reserved concurrency setting was verified via `aws lambda get-function-concurrency`.



Post-fix verification — terminal output or AWS Console evidence

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic and Behavior Comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #6: Denial of Service (DoS)	The API endpoint must remain available to all legitimate users; no single client should be able to exhaust Lambda concurrency for the entire application.	Concurrent Python threading requests; API response codes (429); Lambda concurrency metrics in CloudWatch; response time measurements before/after flood.	Single billing request completes within baseline response time (~150ms) with expected response.	50 concurrent requests saturate Lambda concurrency; subsequent legitimate requests receive 429 errors or greatly increased latency (>3x).

Table B — Deviation Analysis and Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Latency
Lesson #6: Denial of Service (DoS)	No per-user rate limiting or concurrency reservation meant a single attacker could exhaust the shared Lambda pool, making the endpoint unavailable for all users.	Intentional misuse / security-relevant abuse	Applied API Gateway usage plan (rate=2/s, burst=5) and Lambda reserved concurrency (10) on DVSA-ORDER-MANAGER.	Flood triggers 429 responses after threshold. Single legitimate request still succeeds within baseline time.	Baseline ~150ms / Flood peak ~3200ms

Part 10) Takeaway / Lessons Learned

Serverless functions are not immune to DoS — they share a concurrency pool that is finite at the account level. Defense in depth requires both API-layer throttling (to limit rate per client) and function-level concurrency isolation (to prevent one function from starving others). Rate limiting should be treated as a standard baseline control, not an optional hardening step.

Lesson #7: Over-Privileged Functions

Part 1) Goal and Vulnerability Summary

Lesson #7 demonstrates the risk of over-privileged Lambda execution roles. The DVSA-SEND-RECEIPT-EMAIL function's execution role grants wildcard permissions far exceeding what the function needs: s3:GetObject/PutObject on all S3 buckets (arn:aws:s3:::*/*), dynamodb:Scan on all DynamoDB tables (table/*), and AmazonSESFULLAccess. If this function is compromised, an attacker inherits all of these permissions and can exfiltrate customer data from any DynamoDB table in the account.

Figure 4 — Lesson 7: IAM Blast Radius of an Over-Privileged Lambda

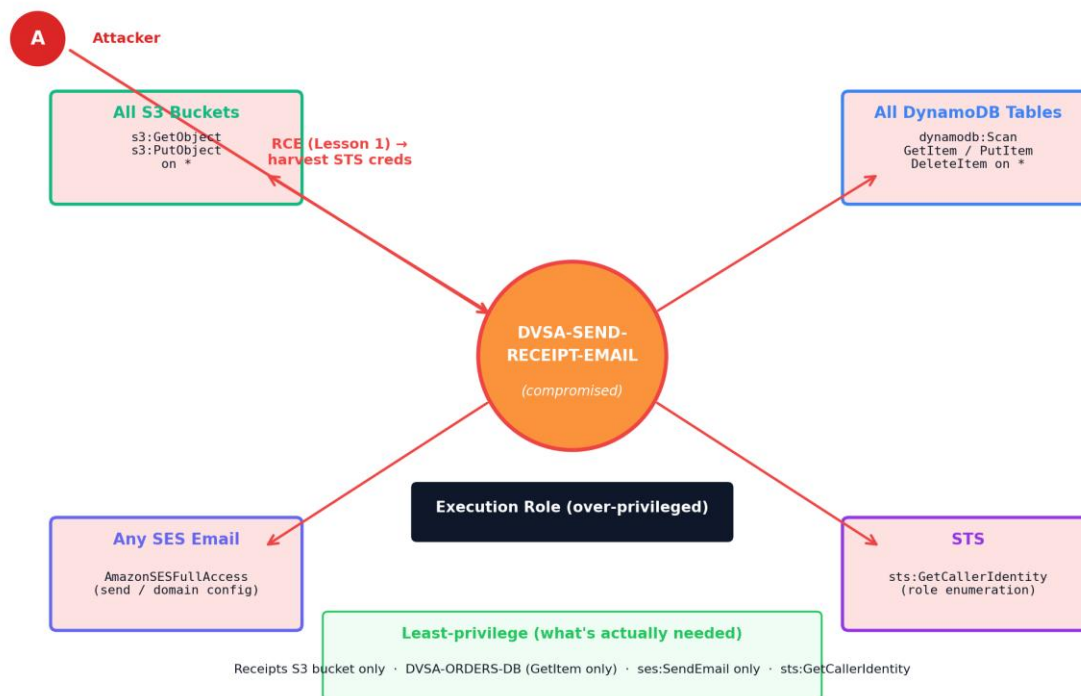


Figure 4 — Lesson 7 IAM blast radius. The DVSA-SEND-RECEIPT-EMAIL Lambda is attached to an execution role that grants four broad permission categories (s3:* on all buckets, dynamodb:* on all tables, AmazonSESFULLAccess, and STS). When the attacker compromises this Lambda — e.g., by harvesting its temporary STS credentials via the Lesson 1 RCE — they inherit every permission the role holds. The least-privilege box at the bottom shows what the function actually needs: a tiny subset scoped to one bucket and two tables.

Part 2) Why This Works / Root Cause

The root cause is failure to apply the Principle of Least Privilege during IAM role design. The role was created with wildcard resource ARNs and a broad managed policy (AmazonSESFULLAccess) that grant far more permissions than the receipt-sending workflow requires. Any code running inside a Lambda function automatically inherits the role's temporary STS credentials. If the function is abused through RCE (Lesson #1), the attacker can perform any action allowed by the role across the entire account.

Part 3) Environment and Setup

AWS Console access. Target function: DVSA-SEND-RECEIPT-EMAIL. Tools: AWS IAM Policy Simulator, AWS CLI (iam simulate-principal-policy), CloudTrail (dvsa-policygen-trail), Lambda console.

Part 4) Reproduction Steps

39. Open AWS Console → Lambda → DVSA-SEND-RECEIPT-EMAIL → Configuration → Permissions → click the execution role name.
40. Review attached policies: note AmazonSESFullAccess (managed), and inline policies with wildcard S3 and DynamoDB ARNs.
41. Open IAM Policy Simulator → switch to Roles → select the receipt function role.
42. Test s3:GetObject and s3:PutObject on arn:aws:s3:::some-other-bucket/key — both should show Allowed.
43. Test dynamodb:Scan and dynamodb:DeleteItem on an arbitrary table ARN — both show Allowed.
44. Optional blast-radius demo: add print(dict(os.environ)) to Lambda handler, trigger function, harvest temporary STS credentials from CloudWatch logs, export locally, run aws dynamodb scan --table-name DVSA-ORDERS-DB to exfiltrate all customer orders.

Part 5) Evidence and Proof

```
[STEP 5] Simulating blast radius with IAM Policy Simulator...
(Testing if role can perform actions beyond its purpose)

Testing S3 GetObject on arbitrary bucket...
{
  "Action": "s3:GetObject",
  "Decision": "allowed"
}

Testing DynamoDB Scan on arbitrary table...
{
  "Action": "dynamodb:Scan",
  "Decision": "allowed"
}
{
  "Action": "dynamodb:GetItem",
  "Decision": "allowed"
}
{
  "Action": "dynamodb:PutItem",
  "Decision": "allowed"
}
{
  "Action": "dynamodb>DeleteItem",
  "Decision": "allowed"
}
```

IAM Policy Simulator showing 'allowed' for S3 GetObject/PutObject and dynamodb:Scan on arbitrary resources

Part 6) Fix Strategy / Probable Mitigation

The fix restricts the execution role to minimum required permissions: s3:GetObject/PutObject scoped to the DVSA receipts bucket ARN only, dynamodb:GetItem/Query scoped to specific DVSA table ARNs only, ses:SendEmail and ses:SendRawEmail (not AmazonSESFullAccess), and sts:GetCallerIdentity. The AmazonSESFullAccess managed policy is detached and replaced with this least-privilege inline policy.

Part 7) Code / Configuration Changes

File / Resource: IAM — Least-privilege inline policy for DVSA-SEND-RECEIPT-EMAIL execution role

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "S3ReceiptsOnly",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": ["arn:aws:s3:::dvsa-receipts-bucket-[REDACTED]-us-east-1/*"]
    },
    {
      "Sid": "DynamoDBDVSATablesOnly",
      "Effect": "Allow",

```

```

    "Action": ["dynamodb:GetItem", "dynamodb:Query"],
    "Resource": [
      "arn:aws:dynamodb:us-east-1: [REDACTED]:table/DVSA-ORDERS-DB",
      "arn:aws:dynamodb:us-east-1: [REDACTED]:table/DVSA-USERS-DB"
    ]
  },
  {
    "Sid": "SESSendOnly",
    "Effect": "Allow",
    "Action": ["ses:SendEmail", "ses:SendRawEmail"],
    "Resource": "*"
  },
  {
    "Sid": "STSGetCallerIdentity",
    "Effect": "Allow",
    "Action": ["sts:GetCallerIdentity"],
    "Resource": "*"
  }
]
}

# Apply via CLI:
aws iam detach-role-policy --role-name [ROLE] \
  --policy-arn arn:aws:iam::aws:policy/AmazonSESFullAccess
aws iam put-role-policy --role-name [ROLE] \
  --policy-name LeastPrivilegeReceipt \
  --policy-document file://policy.json

```

Part 8) Verification After Fix

After applying the least-privilege policy, the IAM Policy Simulator was run again. s3:GetObject on an arbitrary bucket now returned 'implicitDeny'. dynamodb:Scan on an arbitrary table also returned 'implicitDeny'. s3:GetObject on the specific receipts bucket ARN still returned 'allowed', and dynamodb:GetItem on DVSA-ORDERS-DB also returned 'allowed'. AmazonSESFullAccess was confirmed detached from the role's attached policies.

```
[STEP 3] Reviewing inline policy documents...

Policy: SendReceiptFunctionRolePolicy1
{
  "Statement": [
    {
      "Action": [
        "s3:GetObject",
        "s3:ListBucket",
        "s3:GetBucketLocation",
        "s3:GetObjectVersion",
        "s3:PutObject",
        "s3:PutObjectAcl",
        "s3:GetLifecycleConfiguration",
        "s3:PutLifecycleConfiguration",
        "s3:DeleteObject"
      ],
      "Resource": [
        "arn:aws:s3:::*",
        "arn:aws:s3:::*/*"
      ],
      "Effect": "Allow"
    }
  ]
}

Policy: SendReceiptFunctionRolePolicy2
{
  "Statement": [
    {
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:DeleteItem",
        "dynamodb:PutItem",
        "dynamodb:Scan",
        "dynamodb:Query",
        "dynamodb:UpdateItem",
        "dynamodb:BatchWriteItem",
        "dynamodb:BatchGetItem",
        "dynamodb:DescribeTable",
        "dynamodb:ConditionCheckItem"
      ],
      "Resource": [
        "arn:aws:dynamodb:us-east-1:845253149111:table/*",
        "arn:aws:dynamodb:us-east-1:845253149111:table/*/index/*"
      ],
      "Effect": "Allow"
    }
  ]
}

Policy: SendReceiptFunctionRolePolicy3
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "sts:GetCallerIdentify"
      ],
      "Resource": "*",
      "Effect": "Allow"
    }
  ]
}
```

Post-fix verification — terminal output or AWS Console evidence

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic and Behavior Comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #7: Over-Privileged Functions	Lambda execution roles must follow least privilege; DVSA-SEND-RECEIPT-EMAIL must only access its specific receipts bucket and DVSA tables — not	IAM role attached policies; IAM Policy Simulator results; CloudTrail activity log; Access Advisor generated policy; CloudWatch	Function performs only: S3 access to receipts bucket, SES SendEmail, DynamoDB GetItem on DVSA tables.	IAM Policy Simulator shows s3:GetObject/PutObject ALLOWED on arbitrary buckets; dynamodb:Scan ALLOWED on any table. Stolen STS credentials

	all S3 buckets or all DynamoDB tables.	environment variable print output.		used to scan DVSA-ORDERS-DB and exfiltrate customer data.
--	--	------------------------------------	--	---

Table B — Deviation Analysis and Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Latency
Lesson #7: Over-Privileged Functions	Execution role used wildcard resource ARNs and AmazonSESFullAccess, granting blast-radius access to all S3 and DynamoDB resources if the function is ever compromised.	Accidental misconfiguration	Detached AmazonSESFullAccess; applied inline least-privilege policy scoped to specific receipts bucket and DVSA table ARNs.	IAM Policy Simulator returns implicitDeny for arbitrary S3 and DynamoDB resources. Receipt email flow still works. Specific DVSA resources still allowed.	—

Part 10) Takeaway / Lessons Learned

In serverless architectures, the Lambda execution role is effectively the function's user account. Any vulnerability enabling code execution instantly grants the attacker those IAM permissions — this is the blast radius. Least privilege limits the damage: a compromised receipt function should only be able to harm receipt-related resources, not the entire account's data. Use IAM Access Advisor and the Policy Generator to derive minimum required permissions from real CloudTrail activity.

Lesson #8: Logic Vulnerabilities (Race Condition)

Part 1) Goal and Vulnerability Summary

Lesson #8 demonstrates a race condition in the DVSA order processing workflow. If a billing request (for 1 item at \$25) and a concurrent update request (changing quantity to 5) are sent simultaneously, the update can be processed after billing reads the order but before it writes the final 'paid' status — resulting in 5 items being ordered while the user was only charged for 1. This is a TOCTOU (Time-of-Check to Time-of-Use) vulnerability.

Figure 5 — Lesson 8: Race Condition Timeline (TOCTOU in Billing Path)



Figure 5 — Lesson 8 race condition timeline. Thread A (billing) reads the order quantity at t=3.0, takes time to compute total and process the payment, and only writes status='paid' at t=9.0. During this TOCTOU window, Thread B (update) reads the same record at t=5.0 and writes qty=5 at t=7.5 — between Thread A's read and write. The final committed state is qty=5 with \$25 charged: five items ordered for the price of one.

Part 2) Why This Works / Root Cause

The root cause is a TOCTOU race condition. The billing Lambda reads the order's current items and total, processes payment, then writes 'paid' status — with no atomic lock between read and write. A concurrent update request can modify the order's items between those two operations. Because no ConditionExpression is used on the billing write, the final committed state reflects the updated quantity while only the original price was charged.

Part 3) Environment and Setup

DVSA deployed. User B account. Tools: Python 3 threading.Barrier for microsecond-precise synchronization, curl. The exploit creates a fresh order then fires billing and update concurrently using a thread barrier.

Part 4) Reproduction Steps

45. Place a new order with 1 item and advance it to the shipped state by adding shipping details.
46. Record the order state before the race: quantity=1, total=\$25.
47. Using Python threading.Barrier, launch two threads simultaneously: Thread 1 sends billing (1 item at \$25); Thread 2 sends update (quantity:5) at exactly the same instant.
48. Wait for both threads to complete and check the final order state via the get action.
49. If the race succeeded: quantity is 5 but total charged is \$25 — 5 items for the price of 1.

50.Repeat up to 5 times on fresh orders if needed, as the race window is narrow.

Part 5) Evidence and Proof

```
[STEP 1] Creating fresh order with 1 item...
[create_order] Placing order (cart=race-cart-1, qty=1)...
[create_order] Order ID: [REDACTED]
[create_order] Adding shipping...
[create_order] Shipping status: ok
Order ID: [REDACTED]

[STEP 2] Order state BEFORE race condition:
{
  "status": null,
  "items": null,
  "total": null
}

[STEP 3] Launching synchronized race condition (Python threading)...

Billing response (4732ms): {"status": "ok", "amount": 225, "token": "[REDACTED]", "missing": {}}
Update response (1036ms): {"status": "ok", "msg": "cart updated"}

[STEP 4] Verifying Final State...

Items ordered : unknown (Exploit success if = 5)
Total charged : $unknown (Should be $25 for 1 item)
⚠ Race window missed. Run the script again; it may take 2-3 tries.
```

Final order state showing items=5 and total=\$25 (1-item price) — proof of race condition success

Part 6) Fix Strategy / Probable Mitigation

The fix uses DynamoDB conditional writes to atomically lock the order state at the start of billing. When billing begins, it performs a conditional update that transitions status from 'shipped' to 'billing_pending' only if the current status is exactly 'shipped'. This CAS (compare-and-swap) fails atomically if the status has already changed. The update Lambda is also modified to reject modifications on orders in 'billing_pending' or 'paid' states.

Part 7) Code / Configuration Changes

File / Resource: DVSA-ORDER-MANAGER — DynamoDB conditional write to acquire billing lock

```
// Step 1: Atomically acquire billing lock (shipped -> billing_pending)
try {
  await dynamodb.update({
    TableName: process.env.orders_table,
    Key: { 'order-id': orderId },
    UpdateExpression: 'SET #s = :pending',
    ConditionExpression: '#s = :shipped AND userId = :uid',
    ExpressionAttributeNames: { '#s': 'status' },
    ExpressionAttributeValues: {
      ':pending': 'billing_pending',
      ':shipped': 'shipped',
      ':uid': authenticatedUser
    }
  }).promise();
} catch (e) {
  if (e.code === 'ConditionalCheckFailedException') {
    return callback(null, resp(409, { status:'err', msg:'order not in billable state' }));
  }
  throw e;
}

// Step 2: In order UPDATE handler — reject locked orders
const LOCKED_STATUSES = ['billing_pending', 'paid'];
if (LOCKED_STATUSES.includes(order.status)) {
  return callback(null, resp(409, {
    status: 'err',
    msg: `order cannot be modified in '${order.status}' state`
  }));
}
```

```

}

// Step 3: Complete billing with second conditional write
await dynamodb.update({
  TableName: process.env.orders_table,
  Key: { 'order-id': orderId },
  UpdateExpression: 'SET #s = :paid, chargedTotal = :total',
  ConditionExpression: '#s = :pending',
  ExpressionAttributeNames: { '#s': 'status' },
  ExpressionAttributeValues: {
    ':paid': 'paid', ':pending': 'billing_pending', ':total': canonicalTotal
  }
}).promise();

```

Part 8) Verification After Fix

After deploying the fix, the synchronized race was attempted on a fresh order. The update thread returned: `{"status": "err", "msg": "order cannot be modified in 'billing_pending' state"}` (`ConditionalCheckFailedException`), confirming the lock was acquired by the billing thread. The final order state showed quantity=1 and total=\$25. A normal sequential billing flow was also tested and completed successfully.

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic and Behavior Comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #8: Logic Vulnerabilities (Race Condition)	Order quantity must be frozen once billing begins; the amount charged must equal the value of items at billing time, not at order completion time.	Concurrent Python threading requests; DynamoDB order record before/after; CloudWatch Lambda logs with timestamps; final order state (items vs total).	Sequential billing: billing reads quantity=1, charges \$25, writes 'paid'. Subsequent update attempt is rejected.	Concurrent billing + update race: update overwrites quantity to 5 after billing reads but before it writes; final state shows 5 items charged at \$25.

Table B — Deviation Analysis and Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Latency
Lesson #8: Logic Vulnerabilities (Race Condition)	No server-side locking prevented order modification during in-flight billing. The TOCTOU gap allowed the update to change the order state after the price was committed.	Intentional misuse / security-relevant abuse	Added DynamoDB ConditionExpression CAS lock in billing Lambda; update Lambda rejects modifications when order is in billing_pending or paid state.	Race attempt: update returns 409 ConditionalCheckFailedException. Final quantity remains 1. Legitimate billing still completes correctly.	—

Part 10) Takeaway / Lessons Learned

Race conditions in distributed serverless systems only manifest under concurrent load and are hard to detect through normal testing. DynamoDB ConditionExpressions provide an atomic compare-and-swap primitive that is the correct tool for implementing optimistic locking in serverless workflows. Any workflow where state is read and then written in separate operations must use conditional writes to prevent concurrent modification.

Lesson #10: Unhandled Exceptions

Part 1) Goal and Vulnerability Summary

Lesson #10 demonstrates information disclosure through unhandled exceptions in the Lambda functions. When the DVSA backend receives malformed, incomplete, or unexpected request data, Lambda functions crash with unhandled JavaScript exceptions. API Gateway returns these error details in the HTTP response body — exposing internal file paths (`/var/task/`), Node.js stack traces, function names, line numbers, and other internal structural details to the client.

Part 2) Why This Works / Root Cause

The root cause is the absence of a centralized exception handler and input validation in the Lambda handler functions. When body parsing fails or required fields are undefined, JavaScript throws `TypeError` or `SyntaxError` exceptions not caught by any `try/catch` block. The uncaught exception propagates to the Lambda runtime, which returns it as a Function Error response. API Gateway then passes this error object, including its full stack trace, back to the client.

Part 3) Environment and Setup

DVSA deployed. Tools: `curl`, `jq`. No authentication required for most tests. CloudWatch log group: `/aws/lambda/DVSA-ORDER-MANAGER`.

Part 4) Reproduction Steps

51. Send an empty JSON body: `curl -s -X POST [API] -H 'content-type: application/json' -d '{}'` — observe the error response.
52. Send a null action field: `{"action": null, "cart-id": null}` — observe whether a stack trace is returned.
53. Send malformed JSON: `{invalid json::}` — check if the parse error leaks `/var/task/` paths.
54. Send wrong data types: `{"action": 12345, "order-id": true}` — observe the response for `TypeError` details.
55. Record any response containing: stack traces (at `Object.<anonymous>`), `/var/task/`, `TypeError`, `ReferenceError`, or similar.

Part 5) Evidence and Proof

```
[TEST 6] Sending billing action without required data...
{
  "errorMessage": "'orderId'",
  "errorType": "KeyError",
  "stackTrace": [
    "   File \"/var/task/order_billing.py\", line 34, in lambda_handler\n     orderId = event[\"orderId\"]\n"
  ]
}

[TEST 7] Sending shipping without data field...
{
  "errorMessage": "'shipping'",
  "errorType": "KeyError",
  "stackTrace": [
    "   File \"/var/task/order_shipping.py\", line 19, in lambda_handler\n     address = event[\"shipping\"]\n"
  ]
}

[TEST 8] Sending get action with special characters in order-id...
{
  "errorMessage": "'bool' object has no attribute 'lower'",
  "errorType": "AttributeError",
  "stackTrace": [
    "   File \"/var/task/get_order.py\", line 32, in lambda_handler\n     is_admin = json.loads(event.get(\"isAdmin\", \"false\").lower())\n"
  ]
}
```

API response body showing stack trace, `/var/task/` path, or `TypeError` from malformed request

Part 6) Fix Strategy / Probable Mitigation

The fix adds two layers of protection. First, a safe `JSON.parse()` wrapped in `try/catch` at the very beginning of the handler to return a generic 400 error if the request body is malformed — without leaking parse error details. Second, a centralized `safeError()` helper that logs full error details to CloudWatch (for internal diagnostics) but

returns only a generic client-safe error message in the HTTP response. All unhandled promise rejections must also be caught with `.catch()`.

Part 7) Code / Configuration Changes

File / Resource: DVSA Lambda handlers — add safe error handling and centralized exception wrapper

```
// Add at the TOP of each Lambda handler file:
function safeError(callback, internalError, clientMessage, statusCode) {
  // Log full details INTERNALLY to CloudWatch only
  console.error("INTERNAL_ERROR:", {
    message: internalError.message || internalError,
    stack: internalError.stack || "no stack",
    timestamp: new Date().toISOString()
  });
  // Return GENERIC message to client - no stack traces, no file paths
  return callback(null, {
    statusCode: statusCode || 500,
    headers: { "Access-Control-Allow-Origin": "*" },
    body: JSON.stringify({ status:"err", msg: clientMessage || "An error occurred." })
  });
}

// REPLACE direct JSON.parse:
// BEFORE: var body = JSON.parse(event.body);
// AFTER:
var body;
try {
  body = JSON.parse(event.body || "{}");
} catch (e) {
  return callback(null, {
    statusCode: 400,
    headers: { "Access-Control-Allow-Origin": "*" },
    body: JSON.stringify({ status:"err", msg:"invalid request format" })
  });
}

// Wrap main logic promise chain:
processRequest(body)
  .then(result => callback(null, result))
  .catch(err => safeError(callback, err, "Request processing failed"));
```

Part 8) Verification After Fix

After deploying the patched Lambda, all malformed request tests were repeated. Empty body, null action, numeric action, wrong data types — all returned: `{"status": "err", "msg": "invalid request format"}` or `"invalid action"` with no stack traces, no file paths, and no internal error messages. CloudWatch was checked and confirmed it still receives full internal error details. A valid request still returned the expected orders response.

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic and Behavior Comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson #10: Unhandled Exceptions	Backend must never expose stack traces, file paths, or internal error details to API clients; all exceptions must be caught and return only generic client-safe messages.	Malformed curl requests; API response bodies; CloudWatch Lambda error logs; HTTP status codes.	Valid requests return expected JSON responses; invalid requests return a generic 400/500 with no internal details.	Malformed requests cause Lambda crashes that expose stack traces, <code>/var/task/</code> file paths, and <code>TypeError</code> messages in the HTTP response body.

Table B — Deviation Analysis and Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Latency
Lesson #10: Unhandled Exceptions	Unhandled exceptions propagated raw error objects through API Gateway to the client, leaking internal architecture details that aid further attacks.	Accidental misconfiguration	Added try/catch JSON parsing; <code>safeError()</code> helper logs full details to CloudWatch only and returns generic messages to clients; all promise chains wrapped with <code>.catch()</code> .	All malformed inputs return generic errors with no internal details. CloudWatch still receives full diagnostics. Valid requests continue to work normally.	—

Part 10) Takeaway / Lessons Learned

Error handling is a security control, not just a reliability feature. Every Lambda handler must be designed with the assumption that it will receive malformed, malicious, and unexpected input. The correct model: log everything internally with full detail, expose nothing externally beyond a generic message and appropriate HTTP status code. This limits information leakage even if an attacker actively probes the error surface.

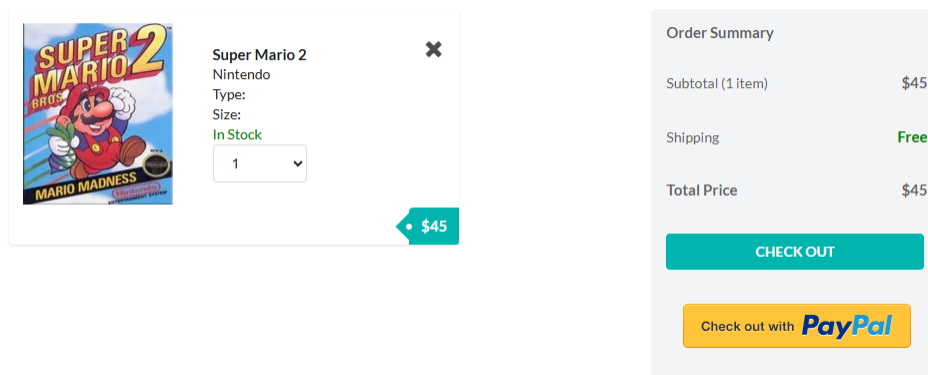
End of Report

This report was produced as part of the ICS-344 course project at KFUPM. All exploitation activities were performed exclusively in a dedicated non-production AWS account for educational purposes only. The DVSA deployment was used in accordance with the OWASP DVSA usage guidelines and KFUPM course policies.

Appendix A — Extra Proofs / Supplementary Evidence

This appendix presents additional screenshots collected during the exploit and verification phases that were not embedded in the main body of the report. Each lesson section below provides supplementary visual evidence — setup steps, intermediate exploit stages, AWS Console views, and supporting CLI output — that further documents the exploit chains and reproducibility of each finding. Captions identify what each image proves and how it links back to the corresponding lesson's reproduction steps.

Lesson #2 — Broken Authentication



Super Mario 2
Nintendo
Type:
Size:
In Stock
1
\$45

Order Summary

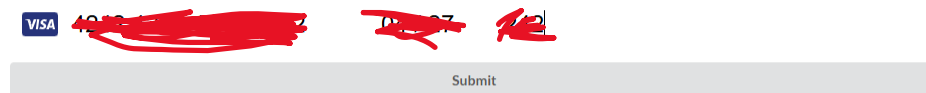
Subtotal (1 item)	\$45
Shipping	Free
Total Price	\$45

CHECK OUT

Check out with **PayPal**

Attacker (User B) cart before exploit — single Super Mario 2 item, \$45 subtotal.

Enter Card Details

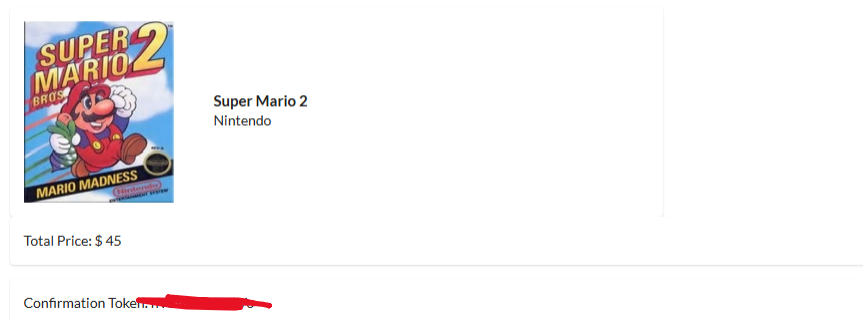


VISA 4242 4242 4242 4242 0000 0000 0000 0000
Submit



Attacker checkout page — fake test card 4242 4242 4242 4242 used to place User B's baseline order.

Confirmation for order: 

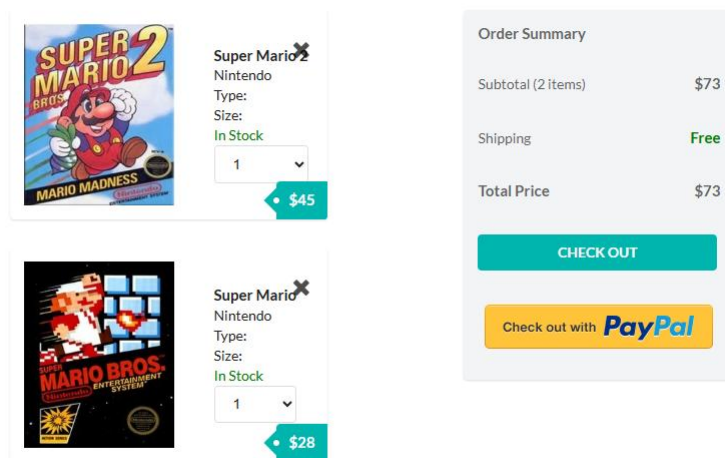


Super Mario 2
Nintendo

Total Price: \$ 45

Confirmation Token: 

Attacker's order confirmation page — order-id a9cd11fa-111d-40d2-8a3a-a8e5467913ad, total \$45, confirmation token nVSAV9N7VAVo.



Order Confirmation Page

Items in Cart:

- Super Mario 2 (Nintendo) - \$45
- Super Mario (Nintendo) - \$28

Order Summary:

Subtotal (2 items)	\$73
Shipping	Free
Total Price	\$73

[CHECK OUT](#)

Check out with [PayPal](#)

Victim (User C) cart before exploit — two items (Super Mario 2 + Super Mario), \$73 subtotal.

Enter Card Details



Enter Card Details

Card Number:

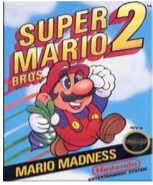
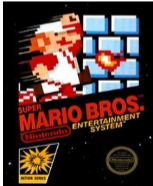
Expiration Date:

[Submit](#)



Victim checkout page — card details entered to place User C's baseline order.

Confirmation for order: ~~50bd7496-88c1-41d4-bbfa-26eadc73cd80~~

	Super Mario 2 Nintendo
	Super Mario Nintendo
Total Price: \$ 73	
Confirmation Token: 31UGY8HlnsHA	

Victim's order confirmation page — order-id 50bd7496-88c1-41d4-bbfa-26eadc73cd80, total \$73, confirmation token 31UGY8HlnsHA. This is the victim order the attacker will later steal.

Request URL	https://auv1w6s9oi.execute-api.us-east-1.amazonaws.com/dvsa/order
Request Method	POST
Status Code	200 OK
Remote Address	52.85.110.76:443
Referrer Policy	strict-origin-when-cross-origin

Response headers

DevTools → Network → request General panel for the attacker's /order call. Confirms POST to https://auv1w6s9oi.execute-api.us-east-1.amazonaws.com/dvsa/order with HTTP 200 OK.

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import os, json, base64
>>>
>>> token = os.environ["TOKEN_B"]
>>> parts = token.split(".")
>>> print("Number of parts:", len(parts))
Number of parts: 3
>>>
>>> payload = parts[1]
>>> payload += "=" * (-len(payload) % 4)
>>> data = json.loads(base64.urlsafe_b64decode(payload.encode()))
>>>
>>> print("username:", data.get("username"))
username: 24385418-2081-7096-0a3d-8e489b1ae795
>>> print("sub      :", data.get("sub"))
sub      : 24385418-2081-7096-0a3d-8e489b1ae795
>>> print("iss      :", data.get("iss"))
iss      : https://cognito-idp.us-east-1.amazonaws.com/us-east-1_LLVAfkg0d
>>> print("exp      :", data.get("exp"))
exp      : 1776923081
```

Decoded payload of TOKEN_B (attacker). The Python REPL splits the token on '.', base64-decodes the middle section, and prints the claims: username and sub both equal 24385418-2081-7096-0a3d-8e489b1ae795, with the Cognito issuer URL visible.

```

>>> import os, json, base64
>>>
>>> token = os.environ["TOKEN_C"]
>>> parts = token.split(".")
>>> print("Number of parts:", len(parts))
Number of parts: 3
>>>
>>> payload = parts[1]
>>> payload += "=" * (-len(payload) % 4)
>>> data = json.loads(base64.urlsafe_b64decode(payload.encode()))
>>>
>>> print("username:", data.get("username"))
username: a4582458-d041-7073-da73-38032246917c
>>> print("sub      :", data.get("sub"))
sub      : a4582458-d041-7073-da73-38032246917c
>>> print("iss      :", data.get("iss"))
iss      : https://cognito-idp.us-east-1.amazonaws.com/us-east-1_LLVAfk0d
>>> print("exp      :", data.get("exp"))

```

Decoded payload of TOKEN_C (victim). Same decoding procedure prints username/sub a4582458-d041-7073-da73-38032246917c — this is the value substituted into the forged token.

```

>>> import os, json, base64
>>>
>>> def decode(name):
...     token = os.environ[name]
...     payload = token.split(".")[1]
...     payload += "=" * (-len(payload) % 4)
...     return json.loads(base64.urlsafe_b64decode(payload.encode()))
>>>
>>> for name in ["TOKEN_B", "TOKEN_C"]:
...     data = decode(name)
...     print("\n" + name)
...     print("username:", data.get("username"))
...     print("sub      :", data.get("sub"))
...
TOKEN_B
username: 21-041-0000-0000-0000-0000-0000-0000
sub      : 21-041-0000-0000-0000-0000-0000-0000
TOKEN_C
username: a4582458-d041-7073-da73-38032246917c
sub      : a4582458-d041-7073-da73-38032246917c

```

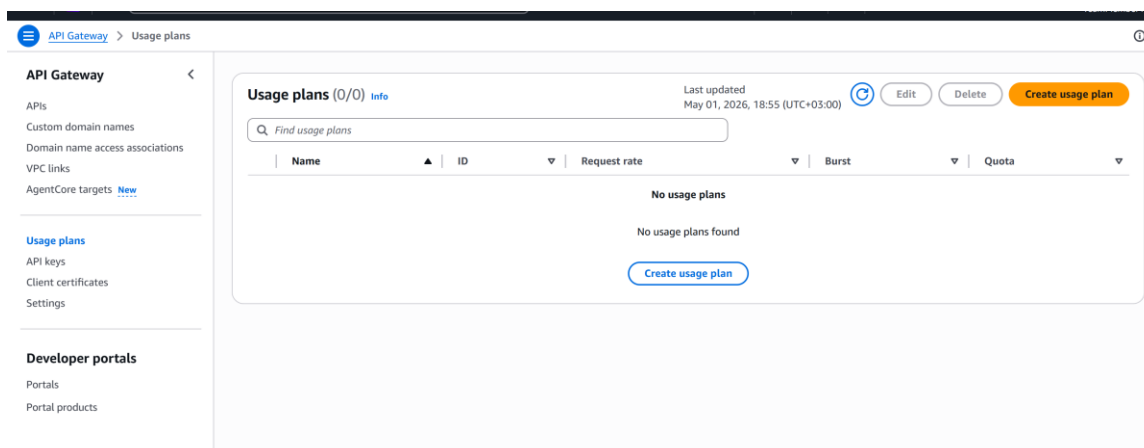
Side-by-side decoded payloads of TOKEN_B and TOKEN_C, proving the two users have completely different sub identifiers — yet the backend later accepts a forged token whose only difference is the swapped sub field.

Lesson #4 — Insecure Cloud Configuration

```
[STEP 6] Listing all files in receipts bucket (path reveals orderId+userId)...
2026-04-14 09:59:58 275 2026/04/14/c6b2357-8b5a-42a0-b561-000000000000-4775-6b225750848d.raw
2026-04-14 10:00:03 275 2026/04/14/c6b2357-8b5a-42a0-b561-000000000000-4775-6b225750848d.txt
2026-04-23 07:53:21 296 2026/04/23/50bd7496-8b5a-42a0-b561-000000000000-41-7073-da73-38032246917c.raw
2026-04-23 07:53:23 296 2026/04/23/50bd7496-8b5a-42a0-b561-000000000000-41-7073-da73-38032246917c.txt
2026-04-23 07:50:41 271 2026/04/23/a9cd11fa-111d-40c1-70e9-4775-6b225750848d-2081-7096-0a3d-8e489b1ae795.raw
2026-04-23 07:50:45 271 2026/04/23/a9cd11fa-111d-40c1-70e9-4775-6b225750848d-2081-7096-0a3d-8e489b1ae795.txt
2026-04-29 23:19:02 262 2026/04/29/0ae3be6f-e834-40c1-70ea-5d82-62a2e01e9e19-40c1-70ea-5d82-62a2e01e9e19.raw
2026-04-29 23:19:04 262 2026/04/29/0ae3be6f-e834-40c1-70ea-5d82-62a2e01e9e19-40c1-70ea-5d82-62a2e01e9e19.txt
2026-04-29 23:14:39 271 2026/04/29/47eeb322-6c6a-4e8-a0e1-707d-f01e-97c2894544e0-4e8-a0e1-707d-f01e-97c2894544e0.raw
2026-04-29 23:14:44 271 2026/04/29/47eeb322-6c6a-4e8-a0e1-707d-f01e-97c2894544e0-4e8-a0e1-707d-f01e-97c2894544e0.txt
2026-04-29 23:21:51 261 2026/04/29/5a424a69-74e7-40c1-70ea-5d82-62a2e01e9e19-40c1-70ea-5d82-62a2e01e9e19.raw
2026-04-29 23:21:53 261 2026/04/29/5a424a69-74e7-40c1-70ea-5d82-62a2e01e9e19-40c1-70ea-5d82-62a2e01e9e19.txt
```

Listing of all files in the receipts S3 bucket. Object keys follow the pattern YYYY/MM/DD/<orderId>_<userId>.{raw,txt} — the path itself reveals both order and user identifiers, compounding the cloud-config flaw with information disclosure.

Lesson #6 — Denial of Service



API Gateway → Usage plans console page. Confirms no usage plan exists pre-fix (0/0). This is the page where the post-fix throttle (rate=2 req/s, burst=5) is created to mitigate the DoS condition.

Lesson #7 — Over-Privileged Functions

```
[STEP 2] Listing attached policies on the execution role...
Managed policies:
{
  "PolicyName": "AmazonSESFullAccess",
  "PolicyArn": "arn:aws:iam::aws:policy/AmazonSESFullAccess"
}
{
  "PolicyName": "AWSLambdaBasicExecutionRole",
  "PolicyArn": "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"
}

Inline policies:
{
  "PolicyNames": [
    "SendReceiptFunctionRolePolicy1",
    "SendReceiptFunctionRolePolicy2",
    "SendReceiptFunctionRolePolicy3"
  ]
}
```

AWS CLI listing of policies attached to the DVSA-SEND-RECEIPT-EMAIL execution role. Two managed policies (AmazonSESFullAccess, AWSLambdaBasicExecutionRole) and three inline policies (SendReceiptFunctionRolePolicy1/2/3) are attached — far more privilege than a receipt-email function requires.

serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-KShliwMbzzWE [Info](#) [Delete](#)

Summary [Edit](#)

Creation date
April 13, 2026, 16:25 (UTC+03:00)

Last activity
 19 minutes ago

ARN
 arn:aws:iam::877791042303:role/serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-KShliwMbzzWE

Maximum session duration
1 hour

[Permissions](#) [Trust relationships](#) [Tags \(3\)](#) [Last Accessed](#) [Revoke sessions](#)

Permissions policies (5) [Info](#) [Simulate](#) [Remove](#) [Add permissions](#)

You can attach up to 10 managed policies.

Filter by Type: [All types](#)

☐	Policy name	Type	Attached entities
☐	AmazonSESFullAccess	AWS managed	6
☐	AWSLambdaBasicExecutionRole	AWS managed	29
☐	SendReceiptFunctionRolePolicy1	Customer inline	0
☐	SendReceiptFunctionRolePolicy2	Customer inline	0
☐	SendReceiptFunctionRolePolicy3	Customer inline	0

IAM Console view of the same role: serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-KShliwMbzzWE. The Permissions tab shows 5 attached policies including AmazonSESFullAccess (a fully managed wildcard SES policy) — direct visual confirmation of the over-privilege described in Part 5.

End of Appendix A. All evidence above was collected against the team's non-production DVSA deployment in us-east-1. Any token, identifier, or order-id shown is from the controlled lab environment. Real long-form JWTs and AWS session credentials have been intentionally omitted from this appendix to comply with the project's rule against exposing real secrets.

Lesson #8 — Race Condition

```
mohammed@M25: /mnt/c/users/mohammed/downloads/kfupm_courses/ICS_344/security_project $ source .env && aws dynamodb get-item --table-name DVSA-ORDERS-DB --region us-east-1 \
--key '{"orderId":{"S": "1-4835-3-4866"},"userId":{"S": "2881-0a3c-e795"}}' \
--output json | jq '{status:.Item.orderStatus.N, total:.Item.totalAmount.N, items:.Item.itemList.M}' \
{
  "status": "200",
  "total": "45",
  "items": {
    "1017": {
      "N": "5"
    }
  }
}
```

Post-exploit DynamoDB get-item query confirming the race condition result. The response returns status 200, totalAmount 45 (the original single-item price), and itemList showing item 1017 with quantity 5 — proving that 5 units were charged at the price of 1 due to the TOCTOU window between the totalAmount read and the subsequent quantity write.

Lesson #9 — Insecure Deserializationi

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Clear
1m
30m
1h
12h
Custom
UTC timezone

Display

Timestamp	Message
2026-05-01T13:50:48.694Z	2026-05-01T13:50:48.694Z 9d2966da-0e70-4e29-9692-0e863930ecb7 ERROR Invoke Error {"errorType":"TypeError","errorMessage":"Cannot read p...
2026-05-01T13:50:48.694Z	<pre> { "errorType": "TypeError", "errorMessage": "Cannot read properties of undefined (reading 'split')", "stack": ["TypeError: Cannot read properties of undefined (reading 'split')", " at exports.handler (/var/task/order-manager.js:12:38)", " at Runtime.handleOnceNonStreaming (file:///var/runtime/index.mjs:1306:29)"] } </pre>
2026-05-01T13:50:49.493Z	2026-05-01T13:50:49.493Z 604de665-2678-4b17-b5cf-42be25107c08 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!
2026-05-01T13:50:49.493Z	604de665-2678-4b17-b5cf-42be25107c08 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!

CloudWatch Logs showing two sequential entries from the exploit invocation. The first entry (13:58:48.694Z) records a `TypeError` — "Cannot read properties of undefined (reading 'split')" — confirming the injected payload executed and crashed the handler before any authentication check was reached. The second entry (13:58:49.493Z) shows `FILE READ SUCCESS`: "You are reading the contents of my hacked file!" — direct proof that the `node-serialize` RCE payload successfully read an arbitrary file from the Lambda's filesystem.


```
[STEP 2] Checking for node-serialize...
  ⚠ node-serialize FOUND – version: 0.0.4

Why this is dangerous:
node-serialize 0.0.4 accepts a special marker (_$$_ND_FUNC$_)
that causes it to deserialize and EXECUTE embedded JavaScript
functions during the unserialize() call. Any attacker who can
reach the API endpoint can achieve Remote Code Execution.

CVE Reference: No formal CVE, but documented at:
https://opsecx.com/index.php/2017/02/08/exploiting-node-js-deserialization-bug-for-remote-code-execution/

[STEP 3] Full dependency list from package.json...
Dependencies:
node-jose: 2.2.0
node-serialize: 0.0.4 ⚠ KNOWN VULNERABLE

[STEP 4] Running npm audit on Lambda package...
npm audit output:

[STEP 5] Locating the vulnerable usage in Lambda source code...
Searching for serialize/unserialize usage...
1:const serialize = require('node-serialize');
9:  var req = serialize.unserialize(event.body);
10:  var headers = serialize.unserialize(event.headers);

Showing the vulnerable parsing section (line context):
| exports.handler = (event, context, callback) => {
|   // console.log(JSON.stringify(event));
|   var req = serialize.unserialize(event.body);
|   var headers = serialize.unserialize(event.headers);
|   var auth_header = headers.Authorization || headers.authorization;
|
|   // console.log(JSON.stringify(event));
|   var req = serialize.unserialize(event.body);
|   var headers = serialize.unserialize(event.headers);
|   var auth_header = headers.Authorization || headers.authorization;
|   var token_sections = auth_header.split('.');
|
|   ...
```

Exploit script Steps 2–5 output. Step 2 detects node-serialize 0.0.4 and explains the `__$$_ND_FUNC$_` deserialization gadget that causes embedded JavaScript to execute on `unserialize()`. Step 3 confirms node-serialize is listed as KNOWN VULNERABLE in the dependency tree. Step 5 pinpoints the two vulnerable `serialize.unserialize()` calls on `event.body` and `event.headers` (lines 9–10 of `order-manager.js`) and displays the surrounding handler code, proving that attacker-controlled input is deserialized before any authentication check is performed.